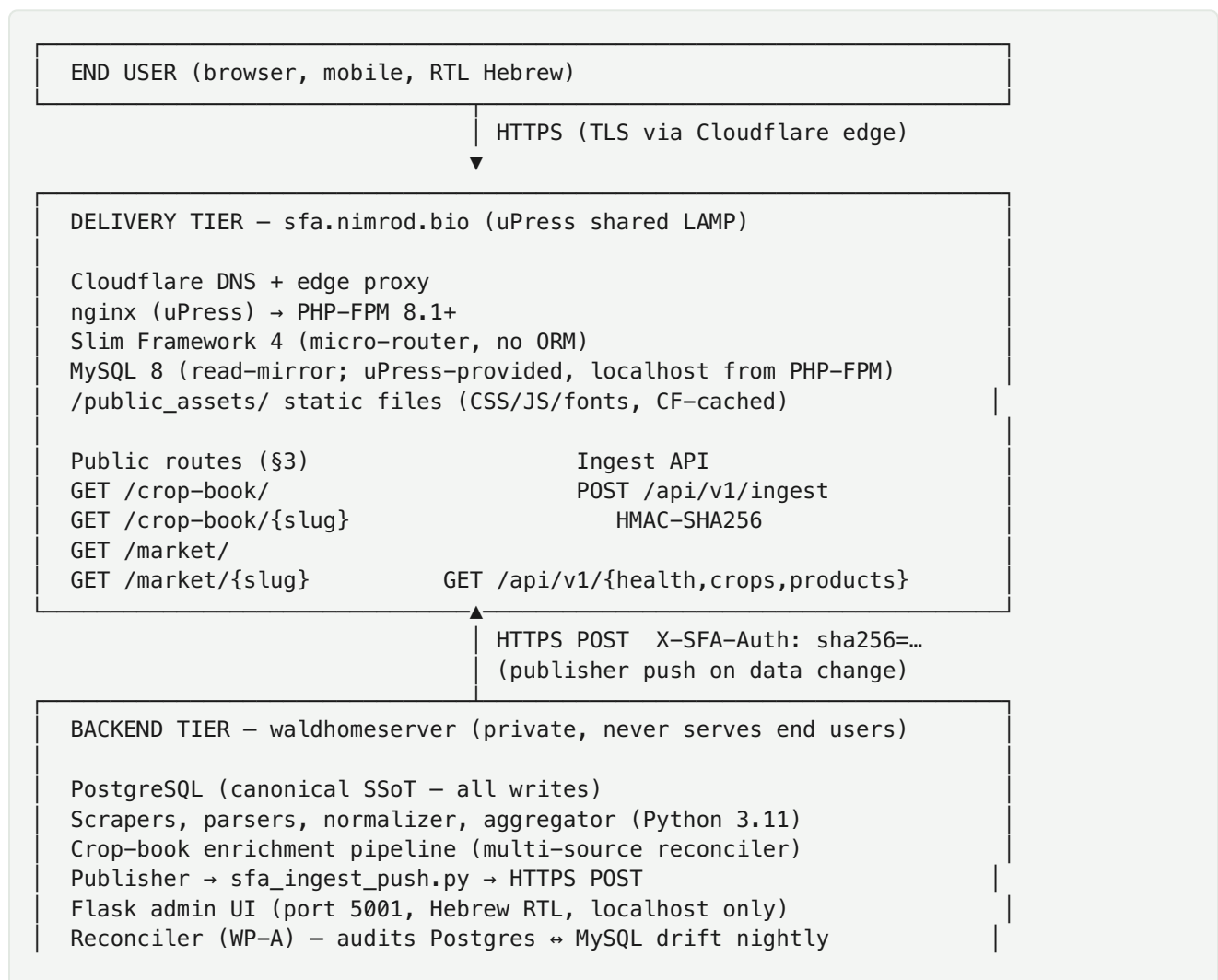


SFA Technical Architecture

Document 06 of the SFA Product Information Pack. Audience: software engineers, platform architects, technical product managers, NotebookLM ingestion. **Sources:** `documentation/02-architecture/sfa-delivery-tier.md`; `documentation/02-architecture/README.md`; `documentation/03-data-and-schema/sfa-mysql-mirror.md`; `organic_market_agent/publisher/sfa_ingest_push.py`; `sfa_delivery/app/Controllers/`; `sfa_delivery/migrations/`; `documentation/09-design-system/README.md`.

Abstract. SmallFarmsAgents (SFA) is built on a strict two-tier architecture: a private backend tier running Python/Flask on waldhomeserver (canonical Postgres SSoT, scrapers, enrichment, publisher) and a public delivery tier running Slim 4/PHP on uPress shared LAMP hosting (`sfa.nimrod.bio` , Cloudflare edge, MySQL read-mirror). The two tiers are decoupled by a narrow, HMAC-authenticated ingest API. This document covers the full end-to-end architecture — request/data flow, the ingest contract, the PHP delivery app internals, the Python backend module structure, the design system, and the extension points that future development will build against.

1. Two-Tier Architecture Overview



Hard invariants: - waldhomeserver NEVER serves end-user HTTP. All public HTTP terminates at the delivery tier. - MySQL on uPress is a read-mirror — writes only via the HMAC-signed ingest API. - Direct MySQL writes from waldhomeserver (bypassing the ingest API) are architecturally forbidden.

2. Tech Stack Reference

Layer	Choice	Rationale
DNS	Cloudflare (existing zone <code>nimrod.bio</code>)	Already in use; edge cache at no extra cost
Edge proxy / TLS	Cloudflare proxied (orange cloud)	Universal SSL auto-provisioned, DDoS shield
Origin web host	uPress shared LAMP	Existing relationship; HTTPS deploy works; MySQL provided
Web server	nginx (uPress-managed)	Provided; <code>.htaccess</code> rewrites handle Slim routing
PHP	8.1+ (uPress default)	Slim 4 requirement
PHP framework	Slim Framework 4 (micro)	~5 MB; just routing + middleware; trivially swappable; no heavy CMS overhead
DB access (PHP)	PDO (PHP-native)	No ORM overhead; portable; short-lived per-request connections
MySQL	uPress-provided (<code>localhost</code> from PHP-FPM)	Standard shared LAMP offering
Migrations (PHP)	Numbered SQL files + 60-line PHP runner (<code>migrate.php</code>)	No Phinx/Doctrine; portable, auditable; sequence is immutable history
Templating	Plain PHP includes	No build step; no framework lock-in
Frontend	Vanilla HTML/CSS/JS (ES5 compatible)	No build step; no Node on uPress; fast; CF-cached
Backend language	Python 3.11	Team fluency; rich ecosystem for NLP, scraping, data
Backend ORM	SQLAlchemy 2.x (mapped column / Mapped API)	Modern declarative ORM; Alembic migrations
Backend DB	PostgreSQL 15/16 (Docker <code>oma-postgres</code>)	ACID; JSON columns; Alembic managed; port 5433
Migrations (Python)	Alembic	60 revisions (001–060); versioned history
Backend framework	Flask (admin UI only)	Hebrew RTL admin; Flask-Login + bcrypt auth
HTTP client (Python)	httpx / requests	Publisher uses <code>requests</code> ; scrapers use both
	Playwright (headless Chromium)	

Layer	Choice	Rationale
Browser automation		Used for certain source collectors requiring JS rendering
Docker	oma-postgres container + docker-compose	Port 5433 (Mac dev); port 5433 (waldhomeserver)
Code deploy	FTPS port 21 + lftp mirror via waldhomeserver relay	uPress FTPS allowlist; Mac Bezeq IP is blocked
Data push	HTTPS POST to /api/v1/ingest (HMAC)	No IP restriction on HTTPS; Mac or server can push

3. Request and Data Flow

3.1 User Page Render (Crop Book detail)

```

User → Cloudflare edge (cache check, TTL 5 min for index pages)
  ↓ cache miss
nginx (uPress) → PHP-FPM → Slim router
  ↓
CropsController::show($slug) or CropBookViewController::show($slug)
  ↓
PDO: SELECT * FROM crops WHERE slug = ?           (1 query)
PDO: SELECT * FROM crop_varieties WHERE crop_id = ? (1 query)
PDO: SELECT * FROM crop_field_enrichment WHERE crop_id = ? (1 query, for calc panel)
PDO: SELECT * FROM crop_attribute WHERE crop_id = ? (1 query, for attribute display)
  ↓
PHP merges top-level columns + payload_json → view data
Template renders Slim PHP view (RTL Hebrew, topic tabs, calculator panels)
  ↓
nginx → Cloudflare edge cache → browser

Typical server-side time: <50 ms
Typical edge-to-browser total: <200 ms

```

3.2 Data Write — Publisher Push

```

Postgres row change detected (last_pushed_at delta in sfa_ingest_push.py)
  ↓
sfa_ingest_push.py builds payload:
{
  "schema_version": 1,
  "table": "crops",                -- or crop_varieties, products, etc.
  "operation": "upsert",
  "idempotency_key": "crops_20260603-143022_001",
  "rows": [ {id, slug, hebrew_name, ..., payload_json: {...}} ]
}
  ↓
HMAC-SHA256(SFA_INGEST_HMAC_SECRET, json_body) → X-SFA-Auth: sha256=<sig>
POST https://sfa.nimrod.bio/api/v1/ingest
  ↓
HmacAuthMiddleware.php (constant-time hash_equals comparison)
  ↓ 200 or 401
IngestController::receive()

```

```

↓
Idempotency check: SELECT FROM ingest_log WHERE idempotency_key = ?
↓ not found → proceed
PDO::beginTransaction()
For each row: upsert into MySQL table (INSERT ... ON DUPLICATE KEY UPDATE)
INSERT INTO ingest_log (idempotency_key, table_name, row_count, status)
PDO::commit()
↓
200 {"accepted": N, "rejected": 0, "idempotency_key": "..."}
↓
sfa_ingest_push.py updates Postgres last_pushed_at only after HTTP 200

```

3.3 Nightly Drift Reconciler

```

waldhomeserver cron → reconciler.py (WP-A team_110)
For each user-facing entity:
  Postgres canonical row → render expected delivery-tier shape
  GET https://sfa.nimrod.bio/api/v1/crops/{slug}
  diff = compare(expected, actual)
  if drift > tolerance:
    queue corrective re-push + log alert (pipeline_alerts table)

```

4. The Ingest API Contract

Endpoint: POST /api/v1/ingest **Authentication:** X-SFA-Auth: sha256=<hmac_hex> (HMAC-SHA256 over the raw JSON body) **Content-Type:** application/json

4.1 Request Envelope

```

{
  "schema_version": 1,
  "table": "<table_name>",
  "operation": "upsert",
  "idempotency_key": "<table>_<YYYYMMDD-HHMMSS>_<seq>",
  "rows": [
    { "<column>": "<value>", ... }
  ]
}

```

Required envelope fields: `schema_version` (must be 1), `table` (from the whitelist), `operation` (upsert or delete), `idempotency_key`, `rows` (array).

4.2 Table Whitelist and Column Allowlists

`IngestController.php` maintains a hard-coded `TABLE_COLUMNS` map. Any column not in the allowlist is silently dropped from the upsert.

Table	Allowed columns
crops	id, slug, hebrew_name, scientific_name, family_id, family_name_he, category, season, dtm_min, dtm_max, last_pushed_at, payload_json

Table	Allowed columns
crop_varieties	id, crop_id, name, payload_json
products	id, slug, hebrew_name, category, unit, last_price, last_price_date, freshness_days, last_pushed_at, payload_json
product_prices	product_id, price_date, price, source
crop_field_enrichment	crop_id, field_name, value_best, unit, field_state, winning_source_class, confidence_score, last_pushed_at
crop_attribute	crop_id, attribute_key, value_canonical, value_list, field_state, last_pushed_at

4.3 Upsert Mechanics

MySQL path (production): `INSERT INTO {table} (...) VALUES (...) ON DUPLICATE KEY UPDATE ...`

Conflict keys: - crops, crop_varieties, products: id (PK) - product_prices: (product_id, price_date, source) unique constraint - crop_field_enrichment: (crop_id, field_name) PK - crop_attribute: (crop_id, attribute_key) unique constraint

Nested JSON values in rows (arrays or objects) are serialized with `json_encode(..., JSON_UNESCAPED_UNICODE)` before binding.

4.4 Idempotency

The ingest log (ingest_log table) records every accepted push. If idempotency_key already exists, the controller returns `{"duplicate": true, "previously_accepted": N}` with HTTP 200 without re-applying. The publisher updates Postgres last_pushed_at only after receiving a non-duplicate 200 response.

Idempotency_key format: `"{table}_{YYYYMMDD-HHMMSS}_{seq}"` (e.g., crops_20260603-143022_001). Batches of 50 rows use sequential seq numbers.

4.5 Response

```
{"accepted": 47, "rejected": 0, "errors": [], "idempotency_key": "crops_20260603-143022_001"}
```

HTTP 400 for malformed envelope. HTTP 401 from HMAC middleware on signature mismatch. HTTP 500 on transaction failure (rolls back).

5. Delivery-Tier PHP App Structure (sfa_delivery/)

```
sfa_delivery/
├── index.php           - Entry point; Slim app bootstrap; all routes defined here
├── .htaccess           - URL rewrites (all non-asset requests → index.php) + security block
├── modules.php         - Module registry (which feature tabs are enabled)
├── composer.json       - Slim 4, PSR-7 Nyholm, PSR-15 middleware, Monolog
├── migrations/         - Numbered SQL files (MySQL DDL; never edit applied files)
│   └── 001_schema_migrations.sql
```

```

├── 002_crops.sql
├── 003_products.sql
├── 004_crop_field_enrichment.sql
├── 005_crop_attribute.sql
├── migrate.php          - Web-accessible migration runner (token-gated; remove after use)
├── app/
│   └── Controllers/
│       ├── CropsController.php          - GET /api/v1/crops, /api/v1/crops/{slug}
│       ├── CropBookViewController.php   - GET /crop-book/, /crop-book/{slug}
│       ├── ProductsController.php       - GET /api/v1/products, /api/v1/products/{slug}
│       ├── MarketViewController.php     - GET /market/, /market/{slug}
│       ├── IngestController.php         - POST /api/v1/ingest (HMAC-authenticated)
│       ├── HubController.php            - Calculator hub; reads crop_field_enrichment
│       ├── AssumptionsController.php    - GET /api/v1/assumptions/{slug}
│       ├── HomeController.php           - GET /
│       ├── ModulesController.php        - Module availability check
│       ├── SearchController.php         - Search endpoints
│       ├── AccountController.php        - User account (S004-reserved)
│       └── HealthController.php         - GET /api/v1/health
├── templates/
│   └── pages/
│       ├── home.php
│       ├── book_crop.php                - Single crop page (13-topic tabs, calc panels)
│       ├── book_grid.php                - Crop book grid (filter: category, season, DTM)
│       ├── market_product.php
│       └── market_index.php
├── public_assets/
│   ├── css/
│   │   └── tokens.css                  - --gj-* design tokens (v2 white-green palette)
│   ├── js/
│   │   └── cropbook-v1.js              - CALC[kind] calculator functions (ES5 vanilla)
│   ├── fonts/                          - Self-hosted: Carmela (licensed), subset web fonts
│   └── img/
│       └── crops/                      - Devora watercolor crop art (web-optimized)
└── data/                              - Static reference data (cover crops, etc.)

```

5.1 FieldRegistry Alias Resolver

The PHP delivery layer includes a `FieldRegistry` alias resolver that maps canonical field names to display labels and handles the canon rename aliases (e.g., `in_row_spacing_cm` → `spacing_in_row_cm`, `days_in_gh_total` → `days_in_nursery`, `avg_yield_per_bed_m` → `yield_per_bed_m`). This ensures the UI renders correctly even during the transition period when some Postgres rows may still use old field names.

5.2 Calculator Hub (`HubController.php`)

The hub controller (`GET /api/v1/hub/calc/{slug}`) executes the PHP-side calculator logic using data from `crop_field_enrichment` (numeric T1 facts) and `crop_attribute` (T2/T3 categoricals). It: 1. Reads the crop's representative-variety enrichment rows. 2. Merges the `ASSUMPTIONS` registry defaults (from the variety's `payload_json.assumptions`). 3. Applies user-supplied assumption overrides from query parameters. 4. Runs the relevant calculators (subset matching the crop's available fields). 5. Returns a JSON response with `{calculator: result, field_state: ..., unavailable: [...]}`.

5.3 Public URL Contract (Frozen)

Path	Method	Auth	Purpose
/	GET	none	Landing page
/crop-book/	GET	none	Crop grid (category / season / DTM filters)
/crop-book/{slug}	GET	none	Single crop (13-topic tabs, calculator panels)
/market/	GET	none	Market index (price index, freshness signal)
/market/{slug}	GET	none	Single product + 30-day price sparkline
/api/v1/health	GET	none	{status, php_version, db: ok/fail, ts}
/api/v1/crops	GET	none	JSON list (pagination, filter params)
/api/v1/crops/{slug}	GET	none	JSON single crop
/api/v1/products	GET	none	JSON list
/api/v1/products/{slug}	GET	none	JSON single product
/api/v1/ingest	POST	HMAC-SHA256	Publisher push from waldhomeserver
/admin/migrate?token=...	GET	One-time token	First-deploy migration runner (removed after use)

Contract frozen 2026-05-23. Changes require a new DECISION artifact.

6. MySQL Mirror Schema

Six tables total (4 data + 2 plumbing). Full DDL in `sfa_delivery/migrations/`.

Strategy: Option B — hybrid minimal top-level columns + `payload_json` blob. Top-level columns are only those needed for `WHERE / ORDER BY` in index queries. Everything else lives in `payload_json` (MySQL JSON column, budgeted ≤ 8 KB per crops row).

Table	Rows (steady state)	Top-level filter cols	Has payload_json?
crops	~70	id, slug, hebrew_name, scientific_name, family_id, family_name_he, category, season, dtm_min, dtm_max, last_pushed_at	yes
crop_varieties	~370	id, crop_id, name	yes
products	~32	id, slug, hebrew_name, category, unit, last_price, last_price_date, freshness_days, last_pushed_at	yes
product_prices	~10k (rolling 90 days)	product_id, price_date, price, source	no (thin time-series)
crop_field_enrichment	~4k (70 crops x ~24 fields)	crop_id, field_name (PK), value_best, unit, field_state, winning_source_class, confidence_score, last_pushed_at	no
crop_attribute			no

Table	Rows (steady state)	Top-level filter cols	Has payload_json?
	~700 (70 crops × ~10 attrs)	crop_id, attribute_key (UNIQUE), value_canonical, value_list, field_state, last_pushed_at	
schema_migrations	(1 per migration)	version, applied_at	no
ingest_log	(rolling 30 days)	idempotency_key (PK), table_name, applied_at, row_count, status	no

payload_json carries schema_version at the top level (required; IngestController rejects payloads missing it). crop_varieties.payload_json embeds the ASSUMPTIONS registry and the field_state map for all whitelisted fields, enabling the JS calculator layer to operate offline after the initial page load.

7. Backend Python Package Structure

organic_market_agent/	– Main Python package
├─ __main__.py	– CLI entry point
├─ admin/	– Flask admin UI (Hebrew RTL)
│ └─ routes/	– 16 Blueprint route modules
│ └─ templates/admin/	– Jinja2 templates (25 templates)
├─ aggregator/	– Daily/weekly rollups + QA + price dispersion rules
├─ collectors/	– HTTP fetch per source type
├─ crop_book/	– Crop Book domain
│ └─ canon/	– Canonical registries (field_registry, units, enums, topics)
│ └─ importer/	– Importers (jmf, tend, reconciler, enrichment_runner, seed)
│ └─ jmf.py	– JMF Excel + MasterClass importer
│ └─ tend.py	– Tend operational records importer
│ └─ reconciler.py	– Multi-source reconciler (pluggable engine)
│ └─ enrichment_runner.py	– Compute + upsert crop_field_enrichment rows
│ └─ ni_importer.py	– NI-class ingestion (skeleton; activates on file arrival)
│ └─ seed.py	– Full seed CLI (--all, --crops, --enrich)
├─ publisher/	– Enrichment JSON publisher
├─ assumptions.py	– ASSUMPTIONS registry (AssumptionField dataclasses)
├─ calculators.py	– 14 pure-function calculators (no I/O)
├─ calculator_meta.py	– Calculator metadata (field dependencies, display labels)
├─ enrichment_models.py	– CropFieldEnrichment SQLAlchemy ORM
├─ attribute_models.py	– CropAttribute SQLAlchemy ORM
├─ field_policy.py	– FieldPolicy + FIELD_POLICY dict (per-field trust config)
├─ source_registry.py	– SOURCE_REGISTRY (7-class SourceSpec dict)
├─ models.py	– Core ORM: CropVariety, CropVarietySourceValue, Crop, etc.
├─ companion_matrix.py	– CropCompanionMatrix ORM
├─ planting_calendar.py	– CropPlantingCalendar ORM
├─ postharvest_storage.py	– CropPostharvestStorage ORM
├─ crop_harvest_stats.py	– CropHarvestStats ORM
└─ crop_knowledge_notes.py	– CropKnowledgeNotes ORM
├─ db/	– DB session, engine, Alembic env
│ └─ versions/	– 60 Alembic migration files (001–060)
├─ maintenance/	– Catalog renormalize, full refresh, prune
├─ models/	– Market pipeline ORM models (14 modules)
├─ normalizer/	– 8-stage price normalizer pipeline + engine
├─ parsers/	– Raw asset → extracted items
└─ publisher/	

	sfa_ingest_push.py	– Crop book + market data push to delivery tier
	engine.py	– Market report publish engine
	rolling_aggregate.py	– 7-day rolling market index
	scheduler/	– Cron-style pipeline orchestration
	utils/	– Config, logging, checksum, alerts, data quality

7.1 Market Pipeline (OrganicMarketAgent)

The market pipeline handles price-index data for ~67 products across ~20 sources (7 active). It runs nightly on waldhomeserver via cron (`scheduler/runner.py`), executing an 8-stage normalizer:

1. **scope_skip** — 301 rules match out-of-scope items (grocery, cleaning, donations) → `ignored`
2. **alias_resolver** — 232 active aliases map raw product names to catalog products (blocking stage)
3. **organic_flag** — detect organic markers in raw product names
4. **price_parser** — extract numeric price from raw text (blocking stage)
5. **unit_resolver** — resolve measurement units from rules + product defaults
6. **quantity_parser** — extract quantity values
7. **price_normalizer** — normalize to canonical unit price
8. **basket_handler** — basket/CSA product handling
9. **confidence** — compute confidence score [0.0, 1.0]

Price dispersion rules suppress publish: 2-source spread >100% triggers

[`AGG_PRICE_RULE:two_source_price_spread_gt_100pct`] ; 3+-source outlier >2 σ triggers

[`AGG_PRICE_RULE:multi_source_outlier_gt_2sigma`] .

8. Design System

The visual design for the SFA public delivery tier is specified in the `team_35 LOD300 handoff` (`_COMMUNICATION/team_35/SFA-S003-P004-WP-CB-1/HANDOFF_PACKAGE/`) and implemented in `sfa_delivery/public_assets/` .

8.1 Design Tokens (`public_assets/css/tokens.css`)

The design uses the AOS Design System v3.4 with `--gj-*` custom property namespacing. The v2 palette is a white-green scheme (five "worlds": leaf, sun, tomato, soil, code). Key token families:

- `--gj-color-leaf-*` — primary green palette (brand color; crop-book primary actions)
- `--gj-color-sun-*` — warm accent (highlight, hover)
- `--gj-color-tomato-*` — error / alert states
- `--gj-color-soil-*` — neutral browns (text on warm backgrounds)
- `--gj-color-code-*` — monospace technical displays
- Confidence/assumption tokens: `--gj-confidence-validated` , `--gj-confidence-unvalidated` , `--gj-confidence-missing` — used for provenance cues in the crop book
- Spacing: `--gj-space-*` (4-point grid)
- Typography: `--gj-font-body` , `--gj-font-heading` , `--gj-font-mono`

8.2 Typography (Brand-Locked)

Font	Role	Delivery
Assistant	Body text (Hebrew RTL, Latin fallback)	Google Fonts (self-hosted subset)
Frank Ruhl Libre	Headings, crop names (Hebrew weight contrast)	Google Fonts (self-hosted subset)
JetBrains Mono	Technical LTR values (numeric data, calculator outputs)	Self-hosted
Carmela	Wordmark + hero text (licensed)	Self-hosted via <code>public_assets/fonts/</code>

8.3 Visual Components

The component library (`cropbook-v1.css`) defines CSS classes for: - `.cb-*` — crop book grid and detail components (card, header, badge, tab) - `.cv-*` — calculator view components (CalcPanel, result display) - `.af-*` — AssumptionField components (editable inline assumption with default display)

The `AssumptionField` component is the key interaction pattern: a numeric input with a pre-set default value, a Hebrew inline explainer, and an optional "read more" link. When the user changes a value, `cropbook-v1.js` recalculates the dependent calculators in real time.

8.4 Crop Art

Devora watercolor crop illustrations (`public_assets/img/crops/`), rendered via `mix-blend-mode: multiply` on near-white backgrounds to integrate cleanly with the white-green palette. Source masters are in the `team_35` handoff; web-optimized copies are in the delivery repo.

8.5 RTL Hebrew Layout

The delivery tier is Right-to-Left Hebrew throughout. CSS uses logical properties (`margin-inline-start` , `padding-inline-end`) rather than physical left/right properties. The `<html dir="rtl" lang="he">` attribute is set on every page. Template rendering uses Hebrew variable names in data structures (field labels, taxonomy topic labels).

8.6 JavaScript Calculator (`cropbook-v1.js`)

The `CALC` object in `public_assets/js/cropbook-v1.js` implements all 14 calculator formulas in vanilla ES5. It must stay in mathematical parity with `organic_market_agent/crop_book/calculators.py` . Known open item: the JS `CALC.revenue` does not yet apply non-kg unit conversions (via `kg_per_unit` from the `unit_size` attribute); the Python `calculators.py` does. This will be resolved once `unit_size` supplies `kg_per_unit` .

9. The Three-Environment Model

The product surfaces two datasets (crop book + market index) with different lifecycles, which flow through different environments:

Environment	Machine	Purpose	Explicitly not
Development	Mac (oma-postgres Docker, port 5433)	Build + curate + enrich crop book; run full Alembic head; generate ingest payloads	Not production; not always-on
Background / pipeline	walldhomeserver (oma-postgres Docker, port 5433)	Run price-index scraping/normalization, ingest-push cron, freshness guard	NOT a staging mirror; never serves end users
Production	uPress sfa.nimrod.bio (Slim/PHP/MySQL)	Serve all end-user HTTP; live MySQL read-mirror	Not where data is authored

Per-dataset canonical sources:

Dataset	Lifecycle	Canonical SSoT	Working store	Publish path
Price index (OMA market)	Dynamic — scraped daily	walldhomeserver Postgres	same	Server cron → HTTPS ingest API (06:30)
Crop book (agronomic knowledge)	Curated / near-static — changes when enriched	git repo (source files + importers + committed WR packs)	Mac oma-postgres (dev)	Mac → HTTPS ingest API on change

Key consequence: the home-server Postgres being at an older Alembic head without the crop-book schema is by design, not a defect. The crop book is a dev→production publish. The home server exists to run processes uPress cannot: long scrapers, Playwright, cron, AOS agents. It does not need to mirror the crop book.

Two transport paths:

Artifact	Transport	Origin	Why
Data (crop book + price index)	HTTPS POST /api/v1/ingest (HMAC) via Cloudflare	Mac OR server	HTTPS to Cloudflare needs no IP allowlist
Code (PHP/CSS/JS)	FTPS port 21 + lftp mirror	walldhomeserver only (relay)	uPress allowlists egress IPs on port 21; Mac's Bezeq IP is blocked

10. Postgres Schema (Backend Canonical)

The canonical Postgres database (Alembic head ~060) contains ~30 tables across two major domains:

Market pipeline tables (core): `sources`, `source_fetch_profiles`, `normalizer_profiles`, `ingestion_runs`, `source_fetch_runs`, `raw_assets`, `raw_extracted_items`, `products`, `product_aliases`, `product_merges`, `catalog_scope_skip_rules`, `normalizer_rules`, `normalized_observations`, `daily_aggregates`, `weekly_snapshots`, `pipeline_alerts`, `scheduler_config`, `users`, `audit_log`, `publish_runs`, `publish_artifacts`, `observation_flags`, `log_entries`, `product_catalog_suggestions`, `pending_product_aliases`.

Crop book tables (added in ~030–060): `crops` , `crop_families` , `crop_varieties` , `crop_variety_source_values` , `crop_field_enrichment` , `crop_attribute` , `crop_planting_calendar` , `crop_harvest_stats` , `crop_postharvest_storage` , `crop_companion_matrix` , `crop_knowledge_notes` , `crop_cover_crops` .

The Alembic migration chain starts at `organic_market_agent/db/versions/001_*.py` . Key migration groups for the crop book: 030–040 (initial crop tables), 041–042 (enrichment layer), 043–058 (attribute layer, canon migration phases), 059 (drop duplicated `crop_varieties` columns), 060 (MIG2 additions — irrigation, pests, labor fields).

11. Flask Admin UI

The Flask admin (`admin/`) runs on port 5001, localhost-only (never exposed to end users). It provides: - Dashboard with KPI funnel (normalized / unresolvable / ignored counts), Chart.js graphs (14-day resolution rate, source success/fail), unread alert panel, maintenance shortcuts. - Source management: per-source stats, observation history, fetch history. - Catalog tools: product list, alias CRUD, scope-skip rules catalog, pending alias inbox. - Pipeline tools: manual run trigger, run detail pages, scheduler control. - Alert viewer: `pipeline_alerts` with mark-read / bulk-read. - Audit log: all write actions logged with user + timestamp.

Authentication: Flask-Login + bcrypt. Session signing key: `ADMIN_SECRET_KEY` env var. UI language: Full Hebrew RTL (Jinja2 templates, shared CSS with the delivery tier).

12. Portability Claim

The delivery tier is a standard LAMP application. Migrating off uPress to any LAMP-capable host:

```
# Snapshot
mysqldump -u $SFA_DB_USER -p $SFA_DB_NAME > snapshot.sql
tar czf code.tgz index.php .htaccess composer.json composer.lock vendor app migrations public_asset

# Upload + restore on new host
scp snapshot.sql code.tgz new-host:/var/www/sfa/
ssh new-host "tar xzf code.tgz && mysql ... < snapshot.sql"

# Point DNS
# Cloudflare → CNAME sfa → new-host
```

Hard architectural invariant: no code on the delivery tier may depend on uPress-specific facilities.

13. Anti-Patterns (Binding)

Anti-pattern	Why forbidden	Use this instead
Direct MySQL writes from waldhomeserver	Bypasses HMAC + idempotency + audit log	Always go through <code>POST /api/v1/ingest</code>

Anti-pattern	Why forbidden	Use this instead
WordPress, plugins, themes on delivery tier	Reintroduces the friction P003 escaped	Pure Slim + PDO
Heavy cron jobs on delivery tier	Shared host — capped CPU	Cron on waldhomeserver; push results
Secrets in code, git, or PHP <code>define()</code>	Leak risk	<code>.env</code> only; <code>chmod 600</code>
ORM (Doctrine, Eloquent)	Heavy deps; obscures SQL	PDO direct
Frontend build step (webpack, vite, npm)	Node not available on uPress	Vanilla HTML/CSS/JS
<code>payload_json</code> field expansion creating new top-level columns	Migrations on delivery should be rare	Add new fields inside <code>payload_json</code> (additive)
Public write endpoints without HMAC or JWT	Trivially abused	HMAC (machine) or JWT (user, S004+)
Long-lived MySQL connections	Shared host connection caps (~50)	Short PDO per request (Slim default)
Reading from <code>payload_json</code> for WHERE/ORDER BY	Slow vs indexed column	Add top-level column + migration if filter is needed

Document 06 — authored 2026-06-03 by team_100 for the SFA Product Information Pack. Sources: *sfa-delivery-tier.md*; *sfa-mysql-mirror.md*; *02-architecture/README.md*; *sfa_ingest_push.py*; *IngestController.php*; *09-design-system/README.md*; *04-pipelines-and-runtime/README.md*.